Frontend

Screens, components, state and design rules

CoolRoute: Pedestrian Routing with Shade and Heat-Exposure Budgets

1. What the frontend must do

- Draw the street map and both routes clearly.
- Let the user change time, budget, shade, step-free and closed footpaths, and update at once.
- Explain the trade-off in words, show a comparison table, and show warnings.
- Work on a phone and on a projector, in light and dark mode.

2. Stack: prototype and production

Layer	Prototype (built)	Production plan
Language and UI	Plain HTML, CSS and JavaScript in one file; no build step	React with TypeScript
Map	SVG drawn from the street graph	MapLibre GL JS with OpenStreetMap-based tiles
Routing engine	JavaScript port of the Python engine, runs in the browser	Calls the backend /route API; same engine on the server
Charts	Inline SVG (sun curve)	Recharts or Chart.js
Styling	CSS variables for colour tokens, light and dark themes	Tailwind CSS with the same tokens
Hosting	Single static file	Free static hosting such as GitHub Pages or Netlify

The prototype is one file so it opens anywhere, even with no internet at the venue. All route calculations happen in the browser.

3. Components

Component	Job	Input	Output
MapView	Draws streets, crossings, missing-data dashes, closed footpaths, A and B pins, both routes	graph, routes, state	SVG
ControlsPanel	Time slider, budget slider, toggles, tap mode, buttons	state	state changes
ResultsCard	Verdict sentence, comparison table, warnings	two routes, budget status	text and table
SunChart	Sun strength through the day with the chosen time marked	hour	SVG
HowItWorks	Formulas in plain words and the 'not a medical risk' note	none	text

4. State

```
state = {
  start: 0, end: 63,        // junction ids
  hour: 13,                 // 5 .. 19, step 0.25
  budget: 60,               // percent of shortest-route heat
  stepFree: false,
  shade: true,              // shade available
  mode: "start" | "end" | "close"   // what a tap on the map does
}
graph.edges[k].closed       // true when the footpath is closed
render() = plan(graph, state) -> draw map -> fill table -> write verdict -> draw chart
```

Every control calls one function, `render()`. There is no hidden state, so what the judge sees is always what the numbers say.

5. Design rules

Rule	How we applied it
Colour has a meaning	Teal = CoolRoute, orange = shortest route, green = shaded street, sand = sunny street, red dotted = closed
Two ways to tell routes apart	Colour and line style: the shortest route is a wide soft halo, CoolRoute is a thin solid line on top
Plain words first	The verdict sentence is above the table, in normal language
Numbers line up	Monospaced digits in the table so columns are easy to compare
Light and dark	All colours are tokens; dark mode redefines the tokens
Phone first	One column below 820 px; 16 px side gutters; tables scroll inside their own box
Honest labels	'Heat exposure' is a score for comparing routes. A note says it is not a medical risk

6. File structure (production)

```
frontend/
  src/
    components/  MapView.tsx  ControlsPanel.tsx  ResultsCard.tsx  SunChart.tsx
    lib/         api.ts  format.ts  sun.ts
    state/       useScenario.ts
    styles/      tokens.css
  public/        manifest.json   (installable PWA)
```

7. Performance and quality

- The engine takes a few milliseconds on 64 junctions and 112 segments, so sliders update live.
- For a real city, the backend does the search and the frontend only draws (see file 04).
- Keyboard focus is visible, controls have labels, and tap targets are large.
- The page was checked for script errors; engine results match Python exactly (file 07).

8. How to run the prototype

- Open `coolroute.html` in any browser. No install, no internet needed except for fonts, which fall back to system fonts.
- The same page is also published as a link in the chat for easy sharing.